

Подробный отчет о выполнении работы по созданию чат-бота.

Данный проект представляет собой чат-бота на языке Python, который предназначен для обработки пользовательских сообщений, определения намерения пользователя и выдачи ответа через набор заранее реализованных навыков.

В проекте реализованы следующие основные возможности:

1. Распознавание намерения пользователя по тексту.
2. Ответ на приветствия.
3. Ответ на прощание.
4. Обработка простого бытового общения.
5. Выдача справки о возможностях бота.
6. Получение информации о погоде.
7. Получение текущего времени.
8. Получение текущей даты.
9. Выполнение простого арифметического сложения.
10. Извлечение города из фразы пользователя.
11. Запоминание состояния диалога, если бот ожидает уточнение города.
12. Запись истории сообщений в базу данных SQLite.
13. Обучение модели классификации интенгов на основе RuBERT.
14. Голосовой ввод через Whisper.
15. Озвучивание ответов через Windows SpeechSynthesizer или Coqui TTS.

Общая структура проекта

Проект состоит из нескольких логических частей.

Файл	Назначение
bot.py	Главный файл запуска ранней версии бота.
router.py	Маршрутизатор интенгов: принимает распознанный интент и вызывает нужный навык.

model_utils.py	Загрузка BERT-модели, токенизатора и выполнение предсказания интента.
train_model.py	Обучение модели классификации интенгов.
label_mapping.json	Словарь соответствия интенгов и числовых классов.
nlp_utils.py	Извлечение города из текста через spaCy.
patterns.py	Регулярные выражения для простых формализованных запросов.
logger.py	Логирование сообщений в SQLite.
voice.py	Голосовой ввод через микрофон и распознавание речи через Whisper.
tts_engine.py	Озвучивание ответа бота.
requirements.txt	Список основных зависимостей проекта.
intents_dataset.csv	Датасет для обучения интент-классификатора.
bot.db	База данных SQLite с логами сообщений.

Интененты, которые распознает бот

Интент — это намерение пользователя. Например, если пользователь пишет «привет», его намерение — поздороваться. Если он пишет «какая погода в Москве», его намерение — узнать погоду.

Интент	Номер класса	Значение
date	0	Запрос текущей даты
goodbye	1	Прощание
greeting	2	Приветствие
help	3	Запрос справки
smalltalk	4	Бытовой разговор
time	5	Запрос текущего времени
weather	6	Запрос погоды

Эти интененты указаны в файле label_mapping.json, где есть два словаря: label2id и id2label. Первый переводит название интенента в номер класса, второй — номер класса обратно в название интенента.

Общий принцип работы бота

Работа бота строится по такой схеме:

1. Пользователь вводит текст или произносит фразу голосом.
2. Если используется голосовой режим, аудио записывается с микрофона.

3. Голос преобразуется в текст через Whisper.
4. Текст передается в обработчик.
5. Сначала могут проверяться точные правила, например регулярное выражение для сложения.
6. Если правило не сработало, текст отправляется в ML-модель.
7. Модель определяет интент и степень уверенности.
8. Интент передается в маршрутизатор.
9. Маршрутизатор вызывает нужный навык.
10. Навык формирует ответ.
11. Ответ выводится пользователю.
12. При необходимости ответ озвучивается.
13. Сообщение пользователя и ответ бота записываются в SQLite-базу.

Таким образом, бот совмещает несколько подходов:

1. Rule-based обработку.
2. Машинное обучение.
3. NLP-анализ.
4. Состояния диалога.
5. Голосовой ввод.
6. Озвучивание ответа.
7. Логирование.

В проекте важную роль играет файл `router.py`. Он принимает уже распознанный интент и решает, какой именно навык нужно вызвать.

В начале файла подключаются отдельные навыки:

1. `from skills.weather import weather_skill`
2. `from skills.time_skill import time_skill`
3. `from skills.date_skill import date_skill`
4. `from skills.greeting_skill import greeting_skill`
5. `from skills.help_skill import help_skill`

6. `from skills.smalltalk_skill import smalltalk_skill`
7. `from skills.goodbye_skill import goodbye_skill`

Это означает, что проект построен по модульной архитектуре: каждый тип действия вынесен в отдельный навык.

Главная функция маршрутизатора называется `route_intent`. Она получает три параметра:

Параметр	Значение
<code>intent</code>	Распознанное намерение пользователя
<code>text</code>	Исходный текст пользователя
<code>user_id</code>	Идентификатор пользователя

Если интент равен `weather`, вызывается `weather_skill`. Если погодный навык возвращает специальное значение `WAIT_CITY`, бот переводит пользователя в состояние ожидания города и задает уточняющий вопрос: «В каком городе вас интересует погода?».

Для остальных интентов маршрутизатор вызывает соответствующие навыки:

Интент	Вызываемая функция
<code>time</code>	<code>time_skill()</code>
<code>date</code>	<code>date_skill()</code>
<code>greeting</code>	<code>greeting_skill()</code>
<code>help</code>	<code>help_skill()</code>
<code>smalltalk</code>	<code>smalltalk_skill()</code>
<code>goodbye</code>	<code>goodbye_skill()</code>

Если интент неизвестен, вызывается `fallback_skill`, который возвращает сообщение: «Я пока не умею обрабатывать такой запрос».

Распознавание интентов через RuBERT

В файле `model_utils.py` задаются пути:

Переменная	Значение
<code>MODEL_DIR</code>	Папка с обученной моделью <code>intent_model</code>
<code>LABEL_MAP_PATH</code>	Файл <code>label_mapping.json</code>

- Функция `load_artifacts` загружает:

1. Токенизатор через `AutoTokenizer.from_pretrained`.
2. Модель классификации через `AutoModelForSequenceClassification.from_pretrained`.

3. Словарь меток из label_mapping.json.

После загрузки модель переводится в режим оценки через `model.eval()`. Это означает, что модель используется не для обучения, а для предсказания.

- Функция `model_is_ready` проверяет, существуют ли папка с моделью и файл словаря меток: `return MODEL_DIR.exists() and LABEL_MAP_PATH.exists()`

То есть бот считает модель готовой только тогда, когда есть и обученная модель, и файл соответствия меток.

- Функция `predict_intent` получает текст пользователя и возвращает два значения:
 1. Название интента.
 2. Уверенность модели.

Если модель или токенизатор не загружены, функция возвращает: `unknown, 0.0`

Далее текст **токенизируется**. Токенизация — это преобразование исходной строки в формат, понятный BERT-модели. В проекте используются параметры:

Параметр	Назначение
<code>return_tensors="pt"</code>	Возврат результата в формате PyTorch
<code>truncation=True</code>	Обрезка слишком длинных текстов
<code>padding=True</code>	Добавление паддинга
<code>max_length=128</code>	Максимальная длина входа — 128 токенов

После токенизации модель делает предсказание. Из выходных значений модели берутся `logits`, затем через `softmax` они превращаются в вероятности классов. Потом выбирается класс с максимальной вероятностью.

После этого номер класса переводится обратно в название интента через `id2label`

Обучение модели

Обучение модели выполняется файлом `train_model.py`. В обучающем скрипте используются:

Библиотека	Назначение
------------	------------

pandas	Чтение и обработка CSV-датасета
numpy	Работа с массивами
torch	Работа с нейросетевой моделью
transformers	Загрузка RuBERT, токенизатора и Trainer
sklearn	Разделение датасета и метрики качества
json	Сохранение словаря меток
pathlib	Работа с путями

Датасет находится в файле `intents_dataset.csv`. Путь к нему задается через переменную `DATASET_PATH`. Также задаются пути к папке модели `intent_model` и файлу `label_mapping.json`. Функция `load_dataset` проверяет наличие файла датасета, затем читает его через `pandas.read_csv` с кодировкой UTF-8. После чтения проверяется, что в датасете есть две обязательные колонки:

Колонка	Назначение
text	Пример пользовательской фразы
intent	Правильный интент

Если этих колонок нет, программа выдает ошибку.

Затем текст и интенты очищаются от лишних пробелов, пустые строки удаляются, а также удаляются строки, содержащие последовательности вида `\x`. Это нужно, чтобы убрать поврежденные строки с неправильной кодировкой или мусорными символами.

Если после очистки осталось меньше 8 строк, обучение не запускается, потому что данных слишком мало.

Функция `build_label_maps` берет список интентов, формирует уникальные названия классов, сортирует их и создает два словаря:

1. `label2id` — название интента → номер.
2. `id2label` — номер → название интента.

Это нужно, потому что нейросетевая модель работает не с текстовыми названиями классов, а с числами.

В функции `main` датасет делится на обучающую и валидационную выборки через `train_test_split`.

Используются параметры:

Параметр	Значение
----------	----------

test_size=0.2	20% данных идут на проверку
random_state=42	Фиксированное случайное разбиение
stratify=df["label"]	Сохранение пропорций классов

Это означает, что модель обучается на 80% примеров, а качество проверяется на оставшихся 20%.

Для обучения используется TrainingArguments. В проекте заданы такие ключевые параметры:

Параметр	Значение	Назначение
num_train_epochs	4	Количество эпох обучения
per_device_train_batch_size	8	Размер батча при обучении
per_device_eval_batch_size	8	Размер батча при проверке
learning_rate	2e-5	Скорость обучения
weight_decay	0.01	Регуляризация
eval_strategy	epoch	Проверка после каждой эпохи
save_strategy	epoch	Сохранение после каждой эпохи
load_best_model_at_end	True	Загрузка лучшей модели после обучения
metric_for_best_model	accuracy	Критерий выбора лучшей модели

После подготовки данных создается объект Trainer, в который передаются:

1. Модель.
2. Аргументы обучения.
3. Обучающий датасет.
4. Валидационный датасет.
5. Функция расчета метрик.

После обучения выполняется предсказание на валидационном датасете. Затем считаются:

1. accuracy.
2. classification_report.

Accuracy показывает долю правильно распознанных сообщений.

classification_report показывает качество отдельно по каждому классу: precision, recall, f1-score, support.

После обучения создается папка intent_model, затем сохраняются:

1. Обученная модель.
2. Токенизатор.

3. Словарь меток label_mapping.json.

Это позволяет при следующем запуске бота не обучать модель заново, а сразу загрузить готовые файлы.

Распознавание города

Для погодных запросов боту нужно понимать, какой город указал пользователь. Например: **Какая погода в Москве?** В этом предложении бот должен выделить город **Москва**.

Для этого используется файл nlp_utils.py и библиотека spaCy. В модуле есть функция get_nlp, которая загружает русскую модель ru_core_news_sm. Чтобы не загружать модель повторно каждый раз, используется кэш _nlp_cache.

Если модель spaCy не установлена, функция выдает понятную ошибку с командой установки:

```
python -m spacy download ru_core_news_sm
```

Функция extract_city работает так:

1. Пытается загрузить русскую модель spaCy.
2. Если загрузить не получилось, возвращает None.
3. Обрабатывает текст пользователя.
4. Ищет именованные сущности с метками LOC или GPE.
5. Если такая сущность найдена, нормализует ее и возвращает как название города.

Нормализация города выполняется через функцию normalize_city. Она проходит по словам найденной сущности, берет леммы и приводит каждое слово к виду с заглавной буквы.

Это нужно, чтобы фраза:

погода в Москве

дала город:

Москва

а не форму:

Москве

Регулярные выражения и арифметика

Помимо ML-классификации, в проекте есть простая rule-based обработка через регулярные выражения.

Файл `patterns.py` содержит список паттернов. Сейчас реализован один паттерн для сложения двух положительных чисел:

- `2+5`
- `2 + 5`
- `3.5 + 4`

При совпадении вызывается функция `handle_addition`.

Такой подход полезен, потому что арифметику не нужно отдавать на распознавание нейросети. Если пользователь ввел строгое выражение, его проще обработать точным правилом.

Однако сейчас поддерживается только сложение. Вычитание, умножение, деление и отрицательные числа в данном паттерне не реализованы.

Логирование в SQLite

Для записи истории работы используется файл `logger.py`.

Функция `init_db` создает таблицу `logs`, если она еще не существует. Таблица содержит поля:

Поле	Назначение
<code>id</code>	Уникальный номер записи
<code>timestamp</code>	Дата и время сообщения
<code>user_message</code>	Сообщение пользователя
<code>bot_message</code>	Ответ бота

Функция `log_message` записывает в таблицу новое сообщение пользователя и ответ бота. Время сохраняется через `datetime.now().isoformat`.

Логирование нужно для нескольких задач:

1. Проверка работы бота.
2. Анализ ошибок.
3. Сбор новых примеров для датасета.

4. Подтверждение того, что бот действительно обрабатывал пользовательские запросы.

В нем применяется стандартная библиотека sqlite3.

Голосовой ввод

OpenAI Whisper требует наличие ffmpeg. В проекте предусмотрена автоматическая подготовка FFmpeg: если системного FFmpeg нет, файл берется через imageio_ffmpeg, копируется в папку проекта как ffmpeg.exe, а путь к нему добавляется в переменную окружения PATH.

Это нужно, чтобы Whisper мог обрабатывать аудиофайлы.

- Запись звука

В проекте используются параметры:

Параметр	Значение
WHISPER_MODEL_NAME	base
INPUT_WAV_PATH	input.wav
SAMPLE_RATE	16000
RECORD_SECONDS	5

Функция `record_audio` записывает звук с микрофона через библиотеку `sounddevice`. Запись идет 5 секунд, используется один канал, частота дискретизации — 16000 Гц. Затем звук преобразуется в формат `int16` и сохраняется в WAV-файл.

- Распознавание речи

Функция `get_whisper_model` загружает модель Whisper только один раз и сохраняет ее в глобальной переменной `_whisper_model`. Это позволяет не загружать модель заново при каждом запросе.

Функция `speech_to_text` принимает путь к WAV-файлу, загружает Whisper-модель и выполняет распознавание речи с параметром `language="ru"`. После распознавания текст очищается.

- Очистка распознанного текста

Функция `clean_text` заменяет ё на е, удаляет лишние символы и оставляет:

1. Русские буквы.
2. Английские буквы.

3. Цифры.
4. Пробелы.
5. Математические знаки +, -, *, /.

- Общая функция голосового ввода

Функция `listen` сначала записывает аудио, затем распознает его и возвращает текст.

То есть голосовой режим работает по схеме:

микрофон → input.wav → Whisper → текст → бот

Озвучивание текста

Озвучивание ответов реализовано в файле `tts_engine.py`.

В проекте используется два способа озвучки:

1. Для русского текста — встроенный голос Windows.
2. Для английского текста — Coqui TTS.

- Определение языка текста

Функция `has_russian_letters` проверяет, есть ли в тексте русские буквы. Если они есть, текст считается русским.

- Нормализация текста для озвучки

Перед озвучиванием текст очищается и подготавливается:

1. Ссылки заменяются на слово ссылка.
2. °C заменяется на градусов Цельсия.
3. км/ч заменяется на километров в час.
4. м/с заменяется на метров в секунду.
5. % заменяется на процентов.
6. Удаляются некоторые Markdown-символы.
7. Лишние пробелы сжимаются.

- Озвучивание русского текста

Для русского текста используется функция `speak_with_windows_voice`. Она вызывает PowerShell, подключает `System.Speech`, создает объект `SpeechSynthesizer` и произносит текст.

Это решение удобно тем, что не требует скачивания отдельной русской нейросетевой TTS-модели.

- Озвучивание английского текста

Для английского текста используется Coqui TTS. В проекте указана модель: `tts_models/en/ljspeech/tacotron2-DDC`

Функция `get_tts` загружает модель Coqui только один раз и затем переиспользует ее.

Для экономии времени используется кэширование WAV-файлов. Текст нормализуется, по нему считается SHA-1 хэш, а затем создается путь к файлу в папке `tts_cache`.

Если файл уже есть в кэше, он используется повторно. Если файла нет, Coqui генерирует новый WAV.

- Выбор способа озвучки

Функция `speak_sync` решает, какой способ использовать:

1. Если в тексте есть русские буквы — Windows voice.
2. Если русских букв нет — Coqui TTS.

Использованные библиотеки

В файле `requirements(1).txt` указаны основные зависимости:

1. `transformers`
`accelerate`
`torch`
`pandas`
`numpy`
`scikit-learn`
`requests`
`spacy`

Основные внешние библиотеки

Библиотека	Назначение
<code>transformers</code>	Работа с RuBERT, токенизатором, Trainer.
<code>accelerate</code>	Ускорение и поддержка обучения моделей Transformers.
<code>torch</code>	PyTorch, основная библиотека для нейросетевой модели.
<code>pandas</code>	Чтение и обработка датасета.
<code>numpy</code>	Работа с массивами, подсчет предсказаний.

scikit-learn	Разделение данных, accuracy, classification_report.
requests	HTTP-запросы, используется для погодного сервиса в соответствующем навыке.
spacy	NLP-обработка русского текста и извлечение города.

Дополнительные библиотеки, которые фактически используются в коде

В новых файлах также используются библиотеки, которых нет в requirements(1).txt:

Библиотека	Где используется	Назначение
TTS	tts_engine.py	Coqui TTS для озвучивания английского текста.
whisper	voice.py	Распознавание речи.
sounddevice	voice.py	Запись звука с микрофона.
scipy	voice.py	Сохранение WAV-файла.
imageio_ffmpeg	voice.py	

Пример полного сценария работы

Рассмотрим сценарий:

Пользователь: Какая погода в Москве?

Шаг 1. Получение текста

Пользователь вводит текст с клавиатуры или произносит его голосом. Если используется голосовой ввод, voice.py записывает звук, сохраняет его в input.wav, затем Whisper преобразует речь в текст.

Шаг 2. Передача текста в модель

Текст передается в predict_intent.

Шаг 3. Токенизация

Токенизатор RuBERT преобразует текст в формат PyTorch с ограничением до 128 токенов.

Шаг 4. Предсказание класса

Модель выдает логиты, они преобразуются в вероятности через softmax, выбирается класс с максимальной вероятностью.

Шаг 5. Получение интента

Номер класса переводится в название интента через id2label. Для данной фразы ожидаемый интент — weather.

Шаг 6. Маршрутизация

Интент weather передается в route_intent, где вызывается weather_skill.

Шаг 7. Извлечение города

Погодный навык должен использовать `extract_city`, чтобы найти город в тексте. Эта функция ищет сущности LOC или GPE через spaCy.

Шаг 8. Получение погоды

Если город найден, погодный навык обращается к погодному сервису и формирует ответ.

Шаг 9. Ответ пользователю

Бот выводит ответ в текстовом виде.

Шаг 10. Озвучка

Если включена озвучка, ответ передается в `speak_sync` или `speak_async`. Русский текст будет озвучен через голос Windows.

Шаг 11. Логирование

Сообщение пользователя и ответ бота записываются в базу `bot.db`.

Что умеет бот

- Приветствие

Бот распознает приветствия и вызывает навык `greeting_skill`.

Примеры запросов:

Привет

Здравствуйте

Добрый день

Интент: `greeting`.

- Прощание

Бот распознает прощание и вызывает `goodbye_skill`.

Примеры:

Пока

До свидания

Увидимся

Интент: `goodbye`.

- Справка

Бот умеет обрабатывать запрос помощи через интент `help`.

Примеры:

Что ты умеешь?
Помощь
Расскажи свои функции
Интент: help.

- Короткие разговоры

Бот умеет отвечать на простые бытовые фразы.

Примеры:

Как дела?
Что нового?
Как настроение?
Интент: smalltalk.

- Дата

Бот умеет обрабатывать запросы текущей даты.

Примеры:

Какая сегодня дата?
Какое сегодня число?
Интент: date.

- Время

Бот умеет обрабатывать запросы текущего времени.

Примеры:

Который час?
Сколько времени?
Интент: time.

- Погода

Бот умеет обрабатывать погодные запросы.

Примеры:

Какая погода в Москве?
Погода в Самаре
Что с погодой в Петербурге?
Интент: weather.

Если город не указан, погодный навык может вернуть WAIT_CITY, после чего маршрутизатор переводит пользователя в состояние ожидания города и задает уточняющий вопрос.

- Сложение

Бот умеет выполнять простое сложение двух положительных чисел через регулярное выражение.

Методы, использованные в проекте

- Классификация интенгов

Главный интеллектуальный метод проекта — классификация интенгов через RuBERT. Бот не просто ищет точное совпадение фразы, а определяет смысл сообщения.

Например, фразы:

Который	час?
Скажи	время
Сколько сейчас времени?	

могут относиться к одному интенгу time.

- Трансформерная модель

Использование RuBERT позволяет учитывать контекст слов в предложении. Это сильнее, чем обычный поиск ключевых слов, потому что модель анализирует фразу целиком.

- Softmax и confidence

После работы модели логиты преобразуются в вероятности через torch.softmax. Затем выбирается класс с максимальной вероятностью, а сама вероятность используется как уверенность модели.

- Маршрутизация интенгов

После распознавания интенга бот не отвечает прямо из модели. Интенг передается в маршрутизатор, который вызывает нужный навык. Это правильная архитектура, потому что ML-модель отвечает только за понимание намерения, а бизнес-логика находится отдельно.

- NLP-извлечение сущностей

Для поиска города используется spaCy и именованные сущности LOC/GPE. Это позволяет находить географические названия внутри обычного текста.

- Rule-based обработка

Для математического сложения используется регулярное выражение. Это пример точного правила, где ML не нужен.

- ASR — распознавание речи

Голосовой ввод реализован через Whisper. Бот записывает звук, сохраняет его в WAV и преобразует речь в текст.

- TTS — синтез речи

Озвучивание ответа реализовано через Windows SpeechSynthesizer для русского текста и Coqui TTS для английского текста.

- Логирование

История общения сохраняется в SQLite. Это позволяет анализировать работу бота и использовать реальные запросы для последующего улучшения датасета.